# MODULE 4

# SQL AND INTEGRITY CONSTRAINT

# CONTENT

- Introduction
- Concept of  DSL.
- Basic Structure
- Set operations
- Aggregate Functions
- Null Values
- Constraint
- Domain Constraints,
- Referential Integrity Constraints
- Assertions, views, Nested Subqueries
- Database security application development using SQL
- Stored procedures and triggers

# INTRODUCTION

- SQL is the Structured Query Language
- It is used to interact with the DBMS
- SQL can
  - Create Schemas in the DBMS
  - Alter Schemas
  - Add data
  - Remove data
  - Change data
  - Access Data

# DSL

- SQL is a Data Sub Language – DSL
- This is a combination of two languages
  - DDL – Data Definition Language
  - DML – Data Manipulation Language

- The main way of accessing data is using the DML command SELECT.
- The abilities of the SELECT command forms the majority of this material on SQL

# SQL BASICS

- Basic SQL statements include
  - CREATE – a data structure
  - SELECT – read one or more rows from a table
  - INSERT – one of more rows into a table
  - DELETE – one or more rows from a table
  - UPDATE – change the column values in a row
  - DROP – a data structure

- In this lecture the focus is on SELECT.

# SIMPLE SELECT

- SELECT column FROM tablename
- SELECT column1,column2,column3 FROM tablename
- SELECT * from tablename
- SELECT * from CAR;

| REGNO | MAKE | COLOUR | PRICE | OWNER |
|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones |
| SC04 BFE | SMART | BLUE | 13000 | |

# SIMPLE SELECT

SELECT regno from CAR;

| REGNO |
| --- |
| F611 AAA |
| J111 BBB |
| A155 BDE |
| K555 GHT |
| SC04 BFE |

# SIMPLE SELECT

SELECT colour,owner from CAR;

| COLOUR | OWNER |
|---|---|
| RED | Jim Smith |
| BLUE | Jim Smith |
| BLUE | Bob Smith |
| GREEN | Bob Jones |
| BLUE | |

# FORMATTING

- SPACES do not matter
- NEWLINES do not matter
- Good practice to put ; at the end of the query.
- CASE (except between single quotes) does not matter.
- These are all valid:

```
SELECT REGNO FROM CAR;
SElecT regno
  From Car
;
```

# COMMENTS

- To give you the ability to make notes in queries you are allowed to have comments.
- Comments are not executed
- A comment starts with -- and ends with a newline
- They are only permitted within a query.

```
SELECT regno -- The registration number
FROM car -- The car storage table
;
```

# SELECT FILTERS

- You can have rules in your queries
- These rules are tested for each row your query produces
- If the rule is true, the row is displayed
- If the rule is false, the row is not displayed
- The rule starts with WHERE

SELECT columns

FROM table

WHERE rule

# SIMPLE RULE

- A simple rule might be to look for a car with a colour of RED.
- The rule would be *colour = 'RED'*

SELECT regno FROM CAR

SELECT regno from CAR
WHERE colour = 'RED'

| REGNO |
|---|
| F611 AAA |
| J111 BBB |
| A155 BDE |
| K555 GHT |
| SC04 BFE |

| REGNO |
|---|
| F611 AAA |

# NOTE

- Things between quotes is CASE SENSITIVE.
- ‘RED’ is not the same as ‘Red’ or ‘red’

- Rules which mention fields – they can be used if they appear on the SELECT line or not.

SELECT regno from CAR
WHERE colour = 'RED'

| REGNO | COLOUR |
|---|---|
| F611 AAA | RED |

# COMPARISONS

- Valid comparisons include =,!=,<>,<,<=,>,>=
  - Colour = ‘RED’ The colour must be red
  - Colour != ‘RED’ The colour is not red
  - Colour <> ‘Red’ Same as !=
  - Price > 10000 More than 10000
  - Price >= 10000 More than or equal to 10000
  - Price < 10000 Cheaper than 10000
  - Price <=10000 Cheaper or the same as 10000
- Numbers – You can say ‘10000’ or 10000. Strings always have quotes…

# DATE

- Date comparisons can be tricky
- You can use all the normal comparators with dates.

SELECT name,dob
from driver

| NAME | DOB |
|---|---|
| Jim Smith | 11 Jan 1980 |
| Bob Smith | 23 Mar 1981 |
| Bob Jones | 3 Dec 1986 |

SELECT name,dob from driver
where DOB = ‘3 Jan 1986’

| NAME | DOB |
|---|---|
| Bob Jones | 3 Dec 1986 |

# DATE

- The tricky part with dates is remembering that dates get bigger as you move into the future.
- DATE1>DATE2 indicates DATE1 is in the future after DATE2.

SELECT name,dob from driver
WHERE DOB >= ‘1 Jan 1981’

| NAME | DOB |
|---|---|
| Bob Smith | 23 Mar 1981 |
| Bob Jones | 3 Dec 1986 |

# Date Syntax

- It must be in quotes
- Each DBMS handles dates in a slightly different way
- Dates like ‘1 Jan 2003’ work quite well.
- Oracle permits dates like ‘1-Jan-2003’
- Oracle also permits dates like ‘1-Jan-03’
  - Be caseful … if you type this it will assume 2003.
  - If you mean 1984 type 1984 not –04.
- You must always specify a day and a month. If you do not the DBMS will report an error.

# BETWEEN

- When dealing with dates sometimes you want to test to see if a field value falls between two dates.
- The easiest way to do this is with BETWEEN

- Find all drivers born between 1995 and 1999

  SELECT name,dob from driver

  WHERE DOB between ‘1 Jan 1985’ and ’31 Dec 1999’
- Between works for other things, not just dates…

  SELECT regno from CAR
  where price between 5000 and 10000;

# NULL

- NULL indicates that something has no value
- It is not a value, and you cannot use normal comparison operators.
- For instance, looking for cars without owners…

Wrong: SELECT regno from car where owner = NULL
Wrong: SELECT regno from car where owner = ‘NULL’

- Instead there are two special operators, IS NULL, and IS NOT NULL

# NULL(CONTD.)

SELECT regno from car
WHERE OWNER is null

| REGNO |
| --- |
| SC04 BFE |

SELECT regno from car
WHERE OWNER is not null

| REGNO |
| --- |
| F611 AAA |
| J111 BBB |
| A155 BDE |
| K555 GHT |
| SC04 BFE |

# LIKE

- Sometimes you want to have a rule involving partial strings, substrings, or wildcards
- LIKE does this, and is a slot-in replacement for ‘=‘
- If the string contains ‘%’ or ‘_’, LIKE uses them to support wildcards.
  - % - Matches 0 or more characters in the string
  - _ - Matches exactly 1 character in the string

# EXAMPLES

- Name LIKE ‘Jim Smith’ e.g. Jim Smith
- Name LIKE ‘_im Smith’ e.g. Tim Smith
- Name LIKE ‘___ Smith’ e.g. Bob Smith
- Name LIKE ‘% Smith’ e.g. Frank Smith
- Name LIKE ‘% S%’ e.g. Brian Smart
- Name LIKE ‘Bob %’ e.g. Bob Martin
- Name LIKE ‘%’ i.e. match anyone

- LIKE is more expensive than =
- If you are not using wildcards, always use = rather than LIKE.

# LOGICAL OPERATORS

- Combining rules in a single WHERE clause would be useful
- AND and OR allow us to do this
- NOT also allows us to modify rule behaviour

- When these are combined together, problems in rule ordering can occur.
- This is solved using parentheses.

# AND

- AND combines rules together so that they ALL must be true.
- Lets revisit the CAR table:

| REGNO | MAKE | COLOUR | PRICE | OWNER |
|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones |
| SC04 BFE | SMART | BLUE | 13000 | |

# AND(CONTD.)

SELECT regno from car
where colour = ‘BLUE’

| REGNO |
| --- |
| J111 BBB |
| A155 BDE |
| SC04 BFE |

SELECT regno from car
WHERE regno LIKE ‘%5’

| REGNO |
| --- |
| A155 BDE |
| K555 GHT |

# AND(CONTD.)

SELECT regno from car
WHERE colour = ‘BLUE’ and regno LIKE ‘%5%’
;

| REGNO |
| --- |
| A155 BDE |

# MULTIPLE AND RULES

- You can have as many rules as you like ANDed together.
- For example:

```
SELECT regno
FROM car
WHERE colour = ‘BLUE’
AND regno like ‘%5%’
AND owner like ‘Bob %’
;
```

# OR

- OR is like ‘either’. So long as one of the rules is true then the filter is true.
- Looks for cars which are EITHER red or blue…

SELECT regno,colour from CAR
WHERE colour = ‘RED’ OR colour = ‘BLUE’

| REGNO | COLOUR |
|---|---|
| F611 AAA | RED |
| J111 BBB | BLUE |
| A155 BDE | BLUE |
| SC04 BFE | BLUE |

# NOT

- NOT inverts the rule it is put in front of:
- WHERE colour = ‘RED’
- This could be inverted as:
  - WHERE colour != ‘RED’
  - WHERE NOT colour = ‘RED’

- NOT is not really useful in this example, but comes into its own in more complex rule sets.

# PRECEDENCE

- Precedence is the order in which the rules are evaluated and combined together.
- It is NOT in the order they are written.
- Rules are combined together firstly at AND, then OR, and finally at NOT.
- Consider : Car has a 5 in reg and is either red or blue.

```
SELECT regno,colour from car
WHERE colour = ‘RED’          -- Line 1
OR    colour = ‘BLUE’         -- Line 2
AND   regno LIKE ‘%5%’        -- Line 3
```

# BRACKETS

- Rewrite as:

```
SELECT regno,colour from car
WHERE (colour = ‘RED’
OR      colour = ‘BLUE’ )
AND     regno LIKE ‘%5%’
```

- Might be clearer as:

```
SELECT regno,colour from car
WHERE ( colour = ‘RED’ OR colour = ‘BLUE’ )
AND     regno LIKE ‘%5%’
```

# DISTINCT

- Find all the colours used in cars.

SELECT colour from car;

| COLOUR |
| --- |
| RED |
| BLUE |
| BLUE |
| GREEN |
| BLUE |

# DISTINCT(CONTD.)

SELECT DISTINCT colour from car;

| COLOUR |
| --- |
| RED |
| BLUE |
| GREEN |

# ORDER BY

- It would be nice to be able to order the output using a sort.
- SELECT make from car;

| MAKE |
| --- |
| FORD |
| SKODA |
| MERCEDES |
| FIAT |
| SMART |

# ASCENDING ORDER

- Sort by alphabetical or numeric order: ASC
- ORDER BY … ASC is the default.

SELECT make from car
ORDER BY make;

| MAKE |
| --- |
| FORD |
| FIAT |
| MERCEDES |
| SKODA |
| SMART |

# DESCENDING ORDER

- Sort by reverse alphabetical or numeric order: DESC
- ORDER BY … DESC must be selected.

SELECT make from car
ORDER BY make DESC;

| MAKE |
| --- |
| SMART |
| SKODA |
| MERCEDES |
| FIAT |
| FORD |

# MULTI COLUMN SORT

- ORDER BY can take multiple columns.

SELECT make,colour FROM car
ORDER BY colour,make;

| MAKE | COLOUR |
|---|---|
| SKODA | BLUE |
| SMART | BLUE |
| MERCEDES | BLUE |
| FIAT | GREEN |
| FORD | RED |

# IN

- When you have a list of OR, all on the same attribute, then IN could be a simpler way:

- Rather Than:

SELECT regno,make FROM car

WHERE make = ‘SKODA’ or make = ‘SMART’

- Have

SELECT regno,make FROM car

WHERE make in (‘SKODA’,’SMART’);

# AGGREGATE FUNCTIONS

- Aggregate functions allow you to write queries to produce statistics on the data in the database.
- These functions are sometimes also called SET functions.

- These include:
  - AVG (calculate the average)
  - SUM
  - MAX
  - MIN
  - COUNT

SELECT price FROM car;

| PRICE |
|---|
| 12000 |
| 11000 |
| 22000 |
| 6000 |
| 13000 |

## AVERAGE

SELECT avg(price) FROM car;

| AVG(PRICE) |
|---|
| 12800 |

## SUM

SELECT sum(price) FROM car;

| SUM(PRICE) |
|---|
| 64000 |

## MAX

- What is the maximum value in a column

  SELECT max(price) FROM car;

| MAX(PRICE) |
| --- |
| 22000 |

## MIN

- What is the minimum value in a column

  SELECT min(price) FROM car;

| MIN(PRICE) |
| --- |
| 6000 |

# COUNT

- How many rows make up a column

SELECT count(price) FROM car;

| COUNT(PRICE) |
| --- |
| 5 |

- Count(*) is similar, but also counts when price is NULL.

SELECT count(*) FROM car;

## COUNT DISTINCT

•Sometimes you do not want to count how many rows are in a column, but how many different values could be found in that column.

•There is a special variant of count which does this:

SELECT count(colour) from car;
SELECT count(DISTINCT colour) from car;

# GROUP BY

- Aggregation functions so far have only been shown in queries with only the single aggregation function on the select line.
- You can combine functions and non-functions on the select line.
- To do this you need GROUP BY.

- Question: What is the most expensive car for each colour.
- Intuitively the following seems right, but will not execute!

```
SELECT colour,max(price)
FROM car;
```

# GROUP BY(CONTD.)

SELECT colour,price
FROM car;

| COLOUR | PRICE |
|---|---|
| RED | 12000 |
| BLUE | 11000 |
| BLUE | 22000 |
| GREEN | 6000 |
| BLUE | 13000 |

SELECT colour,max(price)
FROM car
GROUP BY colour;

| COLOUR | PRICE |
|---|---|
| RED | 12000 |
| BLUE | 22000 |
| GREEN | 6000 |

# HAVING

- WHILE allows rules for each row.
- HAVING allows rules for each group of a GROUP BY.

- Consider the problem “Who has more than 1 car”.

- We would like to say:

SELECT owner from car where count(owner) > 1

- Aggregate functions are not allowed in WHERE.
- They are allowed in HAVING.

# HAVING(CONTD.)

```
SELECT owner,count(regno)
FROM car
GROUP BY owner
HAVING count(regno) > 1
```

OR

```
SELECT owner
FROM car
GROUP BY owner
HAVING count(regno) > 1
```

count(*) works just as well in this case.

# JOIN CONDITIONS

SELECT * from driver;

| NAME | DOB |
|---|---|
| Jim Smith | 11 Jan 1980 |
| Bob Smith | 23 Mar 1981 |
| Bob Jones | 3 Dec 1986 |

# JOIN CONDITIONS

SELECT * from car;

| REGNO | MAKE | COLOUR | PRICE | OWNER |
|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones |
| SC04 BFE | SMART | BLUE | 13000 | |

# SELECT *
# FROM CAR,DRIVER

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Jim Smith | 11 Jan 1980 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Jim Smith | 11 Jan 1980 |
| SC04 BFE | SMART | BLUE | 13000 | | Jim Smith | 11 Jan 1980 |
| F611 AAA | FORD | RED | 12000 | Jim Smith | Bob Smith | 23 Mar 1981 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Bob Smith | 23 Mar 1981 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Smith | 23 Mar 1981 |
| SC04 BFE | SMART | BLUE | 13000 | | Bob Smith | 23 Mar 1981 |
| F611 AAA | FORD | RED | 12000 | Jim Smith | Bob Jones | 3 Dec 1986 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Bob Jones | 3 Dec 1986 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Jones | 3 Dec 1986 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| SC04 BFE | SMART | BLUE | 13000 | | Bob Jones | 3 Dec 1986 |

# FOREIGN KEY LINK

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Jim Smith | 11 Jan 1980 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Jim Smith | 11 Jan 1980 |
| SC04 BFE | SMART | BLUE | 13000 | | Jim Smith | 11 Jan 1980 |
| F611 AAA | FORD | RED | 12000 | Jim Smith | Bob Smith | 23 Mar 1981 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Bob Smith | 23 Mar 1981 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Smith | 23 Mar 1981 |
| SC04 BFE | SMART | BLUE | 13000 | | Bob Smith | 23 Mar 1981 |
| F611 AAA | FORD | RED | 12000 | Jim Smith | Bob Jones | 3 Dec 1986 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Bob Jones | 3 Dec 1986 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Jones | 3 Dec 1986 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| SC04 BFE | SMART | BLUE | 13000 | | Bob Jones | 3 Dec 1986 |

# TRADITIONAL JOIN

```
SELECT *
FROM   car,driver
WHERE owner = name;
```

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |

# MODERN JOIN

```
SELECT *
FROM   car JOIN driver ON ( owner = name )
;
```

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |

# OUTER JOIN

- Consider the last line of the unconstrained join…

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| SC04 BFE | SMART | BLUE | 13000 | | Bob Jones | 3 Dec 1986 |

- This is a car without an owner.
- Sometimes we want to see the rows that fail the join condition due to NULL values.
- To see NULL joins we use OUTER JOIN.
- The JOIN discussed up to this point is known as INNER JOIN.
- We will discuss this more in the relational algebra section.

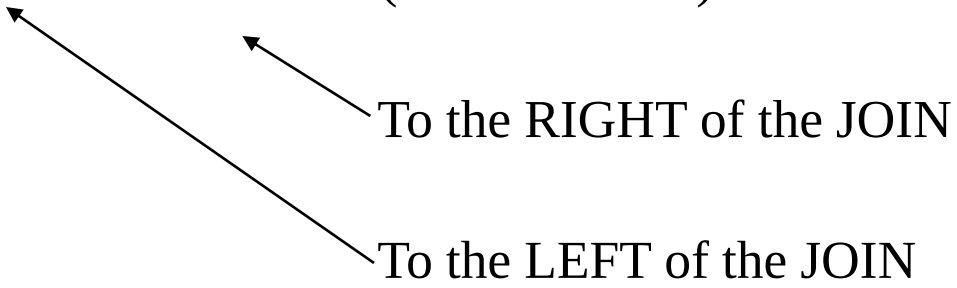
# OUTER JOIN

- Really outer join means we want to force all the rows in one of the tables to appear in the result.
- There are some variants to OUTER JOIN, depending on which rows you want to keep.

# OUTER JOIN

- Consider this:

FROM car JOIN driver on (driver = name)




- If you want all the rows in CAR to always be in the answer, you need a LEFT OUTER JOIN
- If you want all the rows in DRIVER to always be in the answer, you need a RIGHT OUTER JOIN

# OUTER JOIN

- In our example the NULL causing the problem is in OWNER, which is in the table CAR.
- CAR is to the left of the word JOIN
- We need a LEFT OUTER JOIN (which is written LEFT JOIN).

SELECT * FROM car LEFT JOIN driver ON ( owner = name );

# OUTER JOIN

SELECT *FROM car LEFT JOIN driver ON ( owner = name )
;

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| SC04 BFE | SMART | BLUE | 13000 | | | |

# FULL OUTER JOIN

- With a LEFT OUTER join, you keep all the rows from the left.
- With a RIGHT OUTER join, you keep all the rows from the right.
- What if you want to keep all the rows from both sides?
  - You need FULL OUTER join, known as FULL JOIN.
- Consider a new row, David Davis, to DRIVER. David owns no cars.

| NAME | DOB |
|---|---|
| Jim Smith | 11 Jan 1980 |
| Bob Smith | 23 Mar 1981 |
| Bob Jones | 3 Dec 1986 |
| David Davis | 1 Oct 1975 |

# EXAMPLE: LEFT

```
SELECT *
FROM   car LEFT JOIN driver ON ( owner = name );
```

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| SC04 BFE | SMART | BLUE | 13000 | | | |

# EXAMPLE: RIGHT

```
SELECT *
FROM   car RIGHT JOIN driver ON ( owner = name );
```

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| | | | | | David Davis | 1 Oct 1975 |

# EXAMPLE: FULL

```
SELECT *
FROM   car FULL JOIN driver ON ( owner = name )
;
```

| REGNO | MAKE | COLOUR | PRICE | OWNER | NAME | DOB |
|---|---|---|---|---|---|---|
| F611 AAA | FORD | RED | 12000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| J111 BBB | SKODA | BLUE | 11000 | Jim Smith | Jim Smith | 11 Jan 1980 |
| A155 BDE | MERCEDES | BLUE | 22000 | Bob Smith | Bob Smith | 23 Mar 1981 |
| K555 GHT | FIAT | GREEN | 6000 | Bob Jones | Bob Jones | 3 Dec 1986 |
| SC04 BFE | SMART | BLUE | 13000 | | | |
| | | | | | David Davis | 1 Oct 1975 |

# NAMING

- Sometimes column names can be ambiguous.
- Take two tables, called ALPHA and BRAVO. Lets assume each table has a column called NAME.

SELECT name from ALPHA,BRAVO;

- This query fails, as since NAME is in both tables the DBMS cannot work out which NAME you want.
- You can tell the DBMS, by putting the table name in front of the column name, separated by a dot.

SELECT alpha.name from ALPHA,BRAVO;

# Aliases

- If you are writing a big query, you can find yourself typing the same long table names in again and again.
- This can be quite prone to errors.
- SQL allows us to rename tables for the duration of a query.
- You put the new name immediately after the table name in FROM, separated by a space.
- Rather than:

```
SELECT car.owner FROM car;
```

- You can say

```
SELECT c.owner FROM car c;
```

# ALIASES(CONTD.)

- This also works for JOIN.

```
SELECT c.regno, c.owner, d.dob
FROM car c JOIN driver d on (c.owner =
    d.name)
```

- It makes it much easier for the user to work out what values are coming from which table.

# SELFJOIN

- A selfjoin, or an equijoin, is where the same table is used twice in a FROM line.
- It indicates a need to use a single table in two different ways simultaneously.
- Consider the question “Who drives a car the same colour as Bob Smith”?

# SELFJOIN(CONTD.)

SELECT colour FROM car WHERE owner = ‘Bob Smith’;

| Colour |
| --- |
| BLUE |

SELECT owner FROM car
WHERE colour = ‘BLUE’
AND owner != ‘Bob Smith’
AND owner not null;

| owner |
| --- |
| Jim Smith |

# SELFJOIN(CONTD.)

```
SELECT other.owner
FROM car bobsmith, car other
WHERE bobsmith.colour = other.colour       -- 1
AND bobsmith.owner = ‘Bob Smith’   -- 2
AND bobsmith.owner != other.owner  -- 3
AND other.owner NOT NULL           -- 4
;
```

# VIEWS

- VIEWS are to a database what subroutines are to a programming language.
- A view allows us to store a query in the database, so that we can access it later by name.
- We treat views in the same way as a normal table when writing queries.

# VIEWS(CONTD.)

- Lets write a query to tell us how many drivers and how many cars are in the database.
- We could write them separately…

SELECT count(*) from DRIVER;

SELECT count(*) from CAR;

- However this is two queries, and we want to do it in one query…
- Lets store the two queries in two different VIEWs.

# VIEWS(CONTD.)

CREATE VIEW count1 (total) AS select count(*) from driver;
CREATE VIEW count2 (total) AS select count(*) from car;

Select * from count1;

| Total |
|---|
| 3 |

Select * from count2;

| Total |
|---|
| 5 |

Select count1.total,count2.total
from count1,count2;

| Total | Total |
|---|---|
| 3 | 5 |

# Deleting a View

- When you are finished with a VIEW you have created you can delete it.
- You do with with the DROP VIEW command.
- For our example…

DROP VIEW count1;

DROP VIEW count2;

# SUBQUERIES

- Subquery – one select statement inside another
- Used in the WHERE clause
- Subqueries can return many rows.
- Subqueries can only return 1 column.
- Used as a replacement for view or selfjoin.
- Some programmers see them as easier to understand than other options.
- The main drawback is that they can be much slower than selfjoin or view.

# SIMPLE EXAMPLE

- Who in the database is older than Jim Smith?

SELECT dob FROM driver WHERE name = ‘Jim Smith’;

| Dob |
| --- |
| 11 Jan 1980 |

SELECT name FROM driver WHERE dob > ’11 Jan 1980’;

| name |
| --- |
| Bob Smith |
| Bob Jones |

# SIMPLE EXAMPLE

- Combined together:

```
SELECT name
FROM driver
WHERE dob > (SELECT dob
                    FROM driver
                    WHERE name = 'Jim Smith')
;
```

- This query will only work if there is only 1 Jim Smith.

# ANY AND ALL

- To support subqueries which return more than 1 row we need some additional operators… ANY and ALL.
- ANY – changes the rule so that it must be true for at least one of the rows returned from the subquery.
- ALL – changes the rule so that it must be true for each and every row returned from the subquery.

- The ANY or ALL operator goes immediately before the open bracket of the subquery.

# EXAMPLE 1

- What cars are the same solour as a car owned by Jim Smith?
- Jim owns 2 cars, one is RED and the other BLUE. We are looking for cars which are either RED or BLUE.

```
SELECT regno FROM car
WHERE colour = ANY (   SELECT colour
                       FROM car
                       WHERE owner = 'Jim Smith'
                   )
```

# EXAMPLE 2

- List the drivers younger than all the people who own a blue car.
- We are looking for the age of drivers who own a BLUE car, and listing drivers who are younger than all of those ages.

```
SELECT name,dob FROM driver
WHERE dob < ALL (
                    SELECT dob
                    FROM car JOIN driver ON (owner=name)
                    WHERE colour = 'BLUE'
          ) ;
```

# IN AND NOT IN

- We earlier saw IN working for sets like (‘A’,’B’).
- A subquery itself returns its result as a set.
- Therefore we can use IN and NOT IN on subqueries.

- Question: Which cars are the same colour as one of Jim Smith’s cars?

```
SELECT regno FROM car
WHERE colour IN (SELECT colour FROM car
                 WHERE owner = ‘Jim Smith’)
;
```

# EXAMPLE OF NOT IN

- Question: Which cars DO NOT have the same colour as one of Jim Smith's cars?

```
SELECT regno FROM car
WHERE colour NOT IN (SELECT colour
   FROM car
                  WHERE owner = 'Jim Smith')
;
```

# EXISTS

- If a question involves discovering uniqueness, then it can probably be easily solved using the operator EXISTS or NOT EXISTS.
- The EXISTS operator tests the result of running a subquery, and if any rows are returned it is TRUE, else it is FALSE.
- NOT EXISTS does the opposite of EXISTS.

- Note that subqueries can actually refer to tables defined outside the brackets which define the subquery. This is exceptionally useful, but can be slow to execute and confusing to look at.

# EXISTS

- Question: List the colours which are only used once in the database.

```
SELECT colour
FROM car a
WHERE exists (
    SELECT colour                   -- The row does not matter
    FROM car b                      -- unique name from a
    WHERE a.colour = b.colour       -- Same colour as a
    AND a.regno != b.regno          -- Different car from a
);
```

# UNION

- Sometimes you might write two or more queries which produce the same types of answer, and you want to combine the rows of these answers into a single result.
- UNION does this, and basically allows you to join different SELECT statement together.
- UNION automatically removes duplicate rows.

- For the next example, assume a row has been added to the DRIVER table for David Davis, but that he owns no cars.
- Question: List all drivers in the DRIVER table, together with how many cars they own.

# UNION(CONTD.)

SELECT name,count(*)

FROM driver JOIN car on (name = owner)

| NAME | Count(*) |
|---|---|
| Jim Smith | 2 |
| Bob Smith | 1 |
| Bob Jones | 1 |

David Davis is missing, as he did not satisfy the JOIN condition.

# UNION(CONTD.)

- Write a query just for David Davis…

SELECT name,0
FROM driver
WHERE name NOT IN (select owner from car)

| NAME | |
|---|---|
| David Davis | 0 |

# UNION(CONTD.)

- Link the two queries together:

```
SELECT name,count(*)
FROM driver JOIN car on (name = owner)
UNION
SELECT name,0
FROM driver
WHERE name
   NOT IN (select owner
            from car)
```

| NAME | Count(*) |
|---|---|
| Jim Smith | 2 |
| Bob Smith | 1 |
| Bob Jones | 1 |
| David Davis | 0 |

# Changing Data

- We have looked so far at just SELECT
- There are some other useful operators too:
  - INSERT
  - DELETE
  - UPDATE

# INSERT

INSERT INTO table_name
[(column_list)] VALUES (value_list)

The column_list can be omitted if every column is to be assigned a value, otherwise it must list columns to be assigned values. The value_list is a set of literal values giving the value of each column in the same order as the column_list, if specified, or as the columns are defined in the original CREATE TABLE.

insert into driver values (‘Jessie James’,’31 Nov 1892’);
insert into driver (name,dob)values (‘John Johnstone’,’1 Aug 1996’);

# DELETE

DELETE FROM table_name [WHERE condition];

the rows of table_name which satify the condition are deleted.

- Delete Examples:

  DELETE FROM car;  -- Delete all rows from CAR;

  DELETE FROM car
  WHERE owner is null  -- Delete rows for cars without owners;

# UPDATE

UPDATE table_name
SET column_name = expression,{column_name=expression}
[WHERE condition]

Set all BLUE cars to GREEN

UPDATE car SET colour = ‘GREEN’
WHERE colour = ‘BLUE’

Add VAT/Purchase Tax at 17.5% to all prices

UPDATE car SET price = price * 1.175

# View manipulation

When is a view ‘materialised’ or populated with rows of data?

- When it is defined or
- when it is accessed

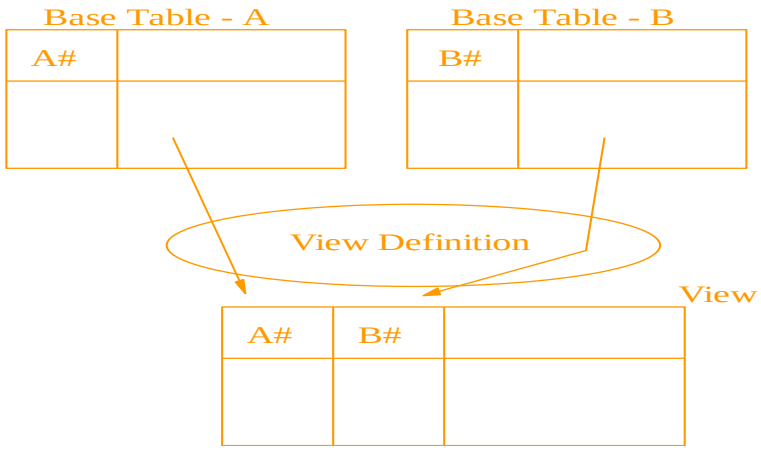
If it is the former then subsequent inserts, deletes and updates would not be visible. If the latter then changes will be seen.

Some systems allow you to chose when views are materialised, most do not and views are materialised whenever they are accessed thus all changes can be seen.

# View update, insert and delete

Can we change views?

- Yes, provided the primary key of all the base tables which make up the view are present in the view.
- This view cannot be changed because we have no means of knowing which row of B to modify



# CONTROLLING SCHEMA

- All the commands so far have allowed data to be looked at, changes, added to, or removed.
- We also need commands to build, change, and remove table definitions.
- We call this schema changes.
- The useful commands to do this include:
  - CREATE TABLE
  - DROP TABLE
  - ALTER TABLE

# CREATE TABLE

- Column types are needed to tell the DBMS what is allowed to be stored in each attribute column.
- A selection of types include:
  - INTEGER
  - REAL
  - DECIMAL : Including DECIMAL(5) and DECIMAL(4,2)
  - VARCHAR
  - CHAR : Pads out strings with spaces
  - DATE

# Syntax

```
CREATE TABLE tablename (
     colname type optionalinfo
    ,colname type optionalinfo
    ,other optional info
);
```

- Optionalinfo could be things like
  - a INTEGER REFERENCES b(c)
  - PRIMARY KEY
  - NOT NULL

# Syntax

- The other optional info at the end of the definition tend to be rules which impact on more than one attribute:
  - PRIMARY KEY (col1,col2,…)
  - FOREIGN KEY (col1,col2,…) REFERENCES othertable

# CAR + DRIVER

```
CREATE table driver (
      name              varchar(30)          PRIMARY KEY,
     dob    date   NOT NULL
);
CREATE TABLE car (
      regno             varchar(8)           PRIMARY KEY,
       make             varchar(20),
       colour           varchar(30),
       price            decimal(8,2),
       owner            varchar(30)          references driver(name)
);

CREATE table driver (
      name     varchar(30),
       dob   date       NOT NULL,
     PRIMARY KEY (name)
);
```

# CAR + DRIVER
# USING ADDITIONAL INFO

```
CREATE TABLE car (
      regno          varchar(8),
       make          varchar(20),
       colour        varchar(30),
       price         decimal(8,2),
       owner         varchar(30),
        FOREIGN KEY(owner),      references driver(name)
     PRIMARY KEY(regno)
);
```

# DROP TABLE

- If you want to delete a table you use DROP TABLE.
  - DROP TABLE tablename
- The main difficulty with dropping a table is referential integrity. As CAR refers to DRIVER, you must delete CAR first then DRIVER. If you try to delete DRIVER first, the system would report an error.

DROP TABLE car;

DROP TABLE driver;

# MODIFY TABLE

- To change a table which already exists you could use MODIFY TABLE.
- It is a complex command with many different options.
- A simple example of it would be adding an address field to the DRIVER table.

ALTER TABLE driver ADD address varchar(50)

# SELECT - ORDER OF EVALUATION

| | |
|---|---|
| SELECT [DISTINCT] column_name | **5,6** eliminate unwanted data |
| FROM label_list | **1** Cartesian Product |
| [WHERE condition ] | **2** eliminate unwanted rows |
| [GROUP BY column_list | **3** group rows |
| [HAVING condition ]] | **4** eliminate unwanted groups |
| [ORDER BY column_list[DESC]] | **7** sort rows |

The last four components are optional.

# DOMAIN CONSTRAINTS

- Integrity constraints guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
- Domain constraints are the most elementary form of integrity constraint.
- They test values inserted in the database, and test queries to ensure that the comparisons make sense.
- New domains can be created from existing data types
  - E.g. **create domain** *Dollars* **numeric**(12, 2)
    **create domain** *Pounds* **numeric**(12,2)
- We cannot assign or compare a value of type Dollars to a value of type Pounds.
  - However, we can convert type as below
    (**cast** *r.A* **as** *Pounds*)
    (Should also multiply by the dollar-to-pound conversion-rate)

# Domain Constraints (Cont.)

- The **check** clause in SQL-92 permits domains to be restricted:
  - Use **check** clause to ensure that an hourly-wage domain allows only values greater than a specified value.

    **create domain** *hourly-wage* **numeric(5,2)**
    **constraint** *value-test* **check**(*value* > = 4.00)
  - The domain has a constraint that ensures that the hourly-wage is greater than 4.00
  - The clause **constraint** *value-test* is optional; useful to indicate which constraint an update violated.
- Can have complex conditions in domain check
  - **create domain** *AccountType* **char**(10)
    **constraint** *account-type-test*
    **check** (**value in** (‘Checking’, ‘Saving’))
  - **check** (*branch-name* **in** (**select** *branch-name* **from** *branch*))

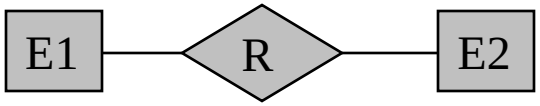
# REFERENTIAL INTEGRITY

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
  - Example: If "Perryridge" is a branch name appearing in one of the tuples in the *account* relation, then there exists a tuple in the *branch* relation for branch "Perryridge".
- Formal Definition
  - Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys $K_1$ and $K_2$ respectively.
  - The subset $\alpha$ of $R_2$ is a ***foreign key*** referencing $K_1$ in relation $r_1$, if for every $t_2$ in $r_2$ there must be a tuple $t_1$ in $r_1$ such that $t_1[K_1] = t_2[\alpha]$.
  - Referential integrity constraint also called subset dependency since its can be written as

$$\prod_{\alpha}(r_2) \subseteq \prod_{K1}(r_1)$$

# Referential Integrity in the E-R Model

- Consider relationship set $R$ between entity sets $E_1$ and $E_2$. The relational schema for $R$ includes the primary keys $K_1$ of $E_1$ and $K_2$ of $E_2$.
  Then $K_1$ and $K_2$ form foreign keys on the relational schemas for $E_1$ and $E_2$ respectively.




- Weak entity sets are also a source of referential integrity constraints.
  - For the relation schema for a weak entity set must include the primary key attributes of the entity set on which it depends

# Checking Referential Integrity on Database Modification

- The following tests must be made in order to preserve the following referential integrity constraint:

$$\prod_{\alpha}(r_2) \subseteq \prod_{K}(r_1)$$

- **Insert.** If a tuple $t_2$ is inserted into $r_2$, the system must ensure that there is a tuple $t_1$ in $r_1$ such that $t_1[K] = t_2[\alpha]$. That is

$$t_2[\alpha] \in \prod_{K}(r_1)$$

- **Delete.** If a tuple, $t_1$ is deleted from $r_1$, the system must compute the set of tuples in $r_2$ that reference $t_1$:

$$\sigma_{\alpha = t1[K]}(r_2)$$

  If this set is not empty
  - either the delete command is rejected as an error, or
  - the tuples that reference $t_1$ must themselves be deleted (cascading deletions are possible).

# Database Modification (Cont.)

- **Update.** There are two cases:
  - If a tuple $t_2$ is updated in relation $r_2$ and the update modifies values for foreign key $\alpha$, then a test similar to the insert case is made:
    - Let $t_2$' denote the new value of tuple $t_2$. The system must ensure that

      $$t_2'[\alpha] \in \prod_K(r_1)$$
  - If a tuple $t_1$ is updated in $r_1$, and the update modifies values for the primary key ($K$), then a test similar to the delete case is made:
    1. The system must compute

       $$\sigma_{\alpha = t1[K]}(r_2)$$

       using the old value of $t_1$ (the value before the update is applied).
    2. If this set is not empty
       1. the update may be rejected as an error, or
       2. the update may be cascaded to the tuples in the set, or
       3. the tuples in the set may be deleted.

# Referential Integrity in SQL

- Primary and candidate keys and foreign keys can be specified as part of the SQL **create table** statement:
  - The **primary key** clause lists attributes that comprise the primary key.
  - The **unique key** clause lists attributes that comprise a candidate key.
  - The **foreign key** clause lists the attributes that comprise the foreign key and the name of the relation referenced by the foreign key.
- By default, a foreign key references the primary key attributes of the referenced table

  **foreign key** (*account-number*) **references** *account*
- Short form for specifying a single column as foreign key

  *account-number* **char** (10) **references** *account*
- Reference columns in the referenced table can be explicitly specified
  - but must be declared as primary/candidate keys

  **foreign key** (*account-number*) **references** *account*(*account-number*)

# Referential Integrity in SQL – Example

**create table** *customer*
*(customer-name* char(20)**,**
*customer-street* char(30),
*customer-city* char(30),
**primary key** (*customer-name))*

**create table** *branch*
(branch-name char(15)**,**
*branch-city* char(30),
*assets* integer,
**primary key** *(branch-name))*

# Referential Integrity in SQL – Example (Cont.)

**create table** *account*
*(account-number* char(10)**,**
*branch-name* char(15),
*balance* integer,
**primary key** (*account-number)*,
**foreign key** (*branch-name)* **references** *branch)*

**create table** *depositor*
*(customer-name* char(20)**,**
*account-number* char(10)**,**
**primary key** *(customer-name*, *account-number)*,
**foreign key** *(account-number)* **references** *account*,
**foreign key** *(customer-name)* **references** *customer)*

# Cascading Actions in SQL

**create table** *account*

. . .
**foreign key***(branch-name)* **references** *branch*
**on delete cascade**
**on update cascade**
. . . **)**

- Due to the **on delete cascade** clauses, if a delete of a tuple in *branch* results in referential-integrity constraint violation, the delete "cascades" to the *account* relation, deleting the tuple that refers to the branch that was deleted.
- Cascading updates are similar.

# Cascading Actions in SQL (Cont.)

- If there is a chain of foreign-key dependencies across multiple relations, with **on delete cascade** specified for each dependency, a deletion or update at one end of the chain can propagate across the entire chain.
- If a cascading update to delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction.
  - As a result, all the changes caused by the transaction and its cascading actions are undone.
- Referential integrity is only checked at the end of a transaction
  - Intermediate steps are allowed to violate referential integrity provided later steps remove the violation
  - Otherwise it would be impossible to create some database states, e.g. insert two tuples whose foreign keys point to each other
    - E.g. *spouse* attribute of relation
      *marriedperson(name, address, spouse)*

# Referential Integrity in SQL (Cont.)

- Alternative to cascading:
  - **on delete set null**
  - **on delete set default**
- Null values in foreign key attributes complicate SQL referential integrity semantics, and are best prevented using **not null**
  - if any attribute of a foreign key is null, the tuple is defined to satisfy the foreign key constraint!

# Assertions

- An *assertion* is a predicate expressing a condition that we wish the database always to satisfy.
- An assertion in SQL takes the form

  **create assertion** <assertion-name> **check** <predicate>
- When an assertion is made, the system tests it for validity, and tests it again on every update that may violate the assertion
  - This testing may introduce a significant amount of overhead; hence assertions should be used with great care.
- Asserting

  for all X, P(X)

  is achieved in a round-about fashion using

  not exists X such that not P(X)

# ASSERTION EXAMPLE

- The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch.

**create assertion** *sum-constraint* **check**
**(not exists (select * from** *branch*
**where (select sum***(amount)* **from** *loan*
**where** *loan.branch-name =*
*branch.branch-name)*
>= **(select sum***(amount)* **from**
*account* **where**
*loan.branch-name = branch.branch-name)))*

# TRIGGERS

- A **trigger** is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.
- Triggers introduced to SQL standard in SQL:1999, but supported even earlier using non-standard syntax by most databases.

# Trigger Example

- Suppose that instead of allowing negative account balances, the bank deals with overdrafts by
  - setting the account balance to zero
  - creating a loan in the amount of the overdraft
  - giving this loan a loan number identical to the account number of the overdrawn account
- The condition for executing the trigger is an update to the *account* relation that results in a negative *balance* value.

# TRIGGER EXAMPLE IN SQL

```
create trigger overdraft-trigger after update on account
referencing new row as nrow
for each row
when nrow.balance < 0
begin atomic
        insert into borrower
           (select customer-name, account-number
            from depositor
            where nrow.account-number =
                        depositor.account-number);
        insert into loan values
           (n.row.account-number, nrow.branch-name,
                                        – nrow.balance);
        update account set balance = 0
        where account.account-number = nrow.account-number
end
```

# Triggering Events and Actions in SQL

- Triggering event can be **insert**, **delete** or **update**
- Triggers on update can be restricted to specific attributes
  - **E.g. create trigger** *overdraft-trigger* **after update of** *balance* **on** *account*
- Values of attributes before and after an update can be referenced
  - **referencing old row as** : for deletes and updates
  - **referencing new row as :** for inserts and updates
- Triggers can be activated before an event, which can serve as extra constraints. E.g. convert blanks to null.

```
create trigger setnull-trigger before update on r
referencing new row as nrow
for each row
    when nrow.phone-number = ' '
    set nrow.phone-number = null
```

# STATEMENT LEVEL TRIGGERS

- Instead of executing a separate action for each affected row, a single action can be executed for all rows affected by a transaction
  - Use **for each statement** instead of **for each row**
  - Use **referencing old table** or **referencing new table** to refer to temporary tables (called ***transition tables***) containing the affected rows
  - Can be more efficient when dealing with SQL statements that update a large number of rows

# External World Actions

- We sometimes require external world actions to be triggered on a database update
  - E.g. re-ordering an item whose quantity in a warehouse has become small, or turning on an alarm light,
- Triggers cannot be used to directly implement external-world actions, BUT
  - Triggers can be used to record actions-to-be-taken in a separate table
  - Have an external process that repeatedly scans the table, carries out external-world actions and deletes action from table
- E.g. Suppose a warehouse has the following tables
  - *inventory(item, level):* How much of each item is in the warehouse
  - *minlevel(item, level) :* What is the minimum desired level of each item
  - *reorder(item, amount):* What quantity should we re-order at a time
  - *orders(item, amount) :* Orders to be placed (read by external process)

# When not to use triggers

- Triggers were used earlier for tasks such as
  - maintaining summary data (e.g. total salary of each department)
  - Replicating databases by recording changes to special relations (called **change** or **delta** relations) and having a separate process that applies the changes over to a replica
- There are better ways of doing these now:
  - Databases today provide built in materialized view facilities to maintain summary data
  - Databases provide built-in support for replication
- Encapsulation facilities can be used instead of triggers in many cases
  - Define methods to update fields
  - Carry out actions as part of the update methods instead of through a trigger

# SECURITY

- **Security** - protection from malicious attempts to steal or modify data.
  - Database system level
    - Authentication and authorization mechanisms to allow specific users access only to required data
    - We concentrate on authorization in the rest of this chapter
  - Operating system level
    - Operating system super-users can do anything they want to the database! Good operating system level security is required.
  - Network level: must use encryption to prevent
    - Eavesdropping (unauthorized reading of messages)
    - Masquerading (pretending to be an authorized user or sending messages supposedly from authorized users)

# Security (cont.)

- Physical level
  - Physical access to computers allows destruction of data by intruders; traditional lock-and-key security is needed
  - Computers must also be protected from floods, fire, etc.
    - More in Chapter 17 (Recovery)
- Human level
  - Users must be screened to ensure that an authorized users do not give access to intruders
  - Users should be trained on password selection and secrecy

# AUTHORIZATION

Forms of authorization on parts of the database:

- **Read authorization -** allows reading, but not modification of data.
- **Insert authorization -** allows insertion of new data, but not modification of existing data.
- **Update authorization -** allows modification, but not deletion of data.
- **Delete authorization -** allows deletion of data

# Authorization (cont.)

Forms of authorization to modify the database schema:

- **Index authorization -** allows creation and deletion of indices.
- **Resources authorization -** allows creation of new relations.
- **Alteration authorization -** allows addition or deletion of attributes in a relation.
- **Drop authorization -** allows deletion of relations.

# Authorization and Views

- Users can be given authorization on views, without being given any authorization on the relations used in the view definition
- Ability of views to hide data serves both to simplify usage of the system and to enhance security by allowing users access only to data they need for their job
- A combination or relational-level security and view-level security can be used to limit a user's access to precisely the data that user needs.

# View Example

- Suppose a bank clerk needs to know the names of the customers of each branch, but is not authorized to see specific loan information.
  - Approach: Deny direct access to the *loan* relation, but grant access to the view *cust-loan*, which consists only of the names of customers and the branches at which they have a loan.
  - The *cust-loan* view is defined in SQL as follows:

```
create view cust-loan as
    select branchname, customer-name
    from   borrower, loan
    where borrower.loan-number = loan.loan-number
```

# Authorization on Views

- Creation of view does not require **resources** authorization since no real relation is being created
- The creator of a view gets only those privileges that provide no additional authorization beyond that he already had.
- E.g. if creator of view *cust-loan* had only **read** authorization on *borrower* and *loan*, he gets only **read** authorization on *cust-loan*

# Security Specification in SQL

- The grant statement is used to confer authorization

  **grant** <privilege list>

  **on** <relation name or view name> to <user list>
- <user list> is:
  - a user-id
  - *public*, which allows all valid users the privilege granted
  - A role (more on this later)
- Granting a privilege on a view does not imply granting any privileges on the underlying relations.
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

# PRIVILEGES IN SQL

- **select:** allows read access to relation,or the ability to query using the view
  - Example: grant users $U_1$, $U_2$, and $U_3$ **select** authorization on the *branch* relation:

    **grant select on** *branch* **to** $U_1$, $U_2$, $U_3$
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **references**: ability to declare foreign keys when creating relations.
- **usage**: In SQL-92; authorizes a user to use a specified domain
- **all privileges**: used as a short form for all the allowable privileges

# Privilege to Grant Privileges

- **with grant option**: allows a user who is granted a privilege to pass the privilege on to other users.
  - Example:

    **grant select on** *branch* **to** $U_1$ **with grant option**

    gives $U_1$ the **select** privileges on branch and allows $U_1$ to grant this privilege to others

# END
# OF
# MODULE 4